

DISCIPLINA: MODELAGEM E PROJETO DE BANCO DE DADOS

Professor: Romes

Atividade Prática — DDL (Data Definition Language)

Aluno(a): GABRIEL NONATO DA SILVA _____ Turma: _____
4° ADS _____ Data: 02 /06 /2026 _____

Instruções

Esta atividade contém 10 questões sobre comandos DDL (Data Definition Language) em SQL. Em cada questão, é apresentado um exemplo de comando DDL aplicado a um contexto. Em seguida, você deve escrever o seu próprio código aplicando o que aprendeu ao tema proposto. Utilize a sintaxe padrão SQL (PostgreSQL/MySQL) e indique tipos de dados, restrições e demais elementos necessários.

Questão 1 — CREATE TABLE — Criação de tabela simples

O comando CREATE TABLE permite criar uma nova tabela no banco de dados, definindo seus atributos, tipos e restrições básicas.

Exemplo:

```
CREATE TABLE cliente (  
    id_cliente INT PRIMARY KEY,  
    nome VARCHAR(100) NOT NULL,  
    email VARCHAR(80) UNIQUE,  
    data_cadastro DATE  
);
```

Sua tarefa: Escreva o comando CREATE TABLE para criar uma tabela chamada produto, contendo: id_produto (chave primária), nome (obrigatório, até 80 caracteres), preço (numérico com 2 casas decimais) e quantidade em estoque (inteiro).

Resposta:

```
1 • ○ CREATE TABLE produto (  
2     id_produto INT PRIMARY KEY,  
3     nome VARCHAR(80) NOT NULL,  
4     preco DECIMAL(10,2),  
5     quantidade_estoque INT  
6 );  
7
```

Questão 2 — CREATE TABLE com PRIMARY KEY composta

Em algumas situações, a chave primária de uma tabela é composta por mais de um atributo, garantindo a unicidade da combinação.

Exemplo:

```
CREATE TABLE item_pedido (  
    id_pedido INT PRIMARY KEY,  
    id_item INT PRIMARY KEY,  
    quantidade INT  
);
```

```
id_pedido INT,  
id_produto INT,  
quantidade INT NOT NULL,  
PRIMARY KEY (id_pedido, id_produto)  
);
```

Sua tarefa: Crie uma tabela chamada matricula com chave primária composta pelos atributos id_aluno e id_disciplina, incluindo também o atributo nota (numérico) e situacao (texto curto).

Resposta:

```
23 • CREATE TABLE matricula (  
24     id_aluno INT,  
25     id_disciplina INT,  
26     nota DECIMAL(4,2),  
27     situacao VARCHAR(20),  
28     PRIMARY KEY (id_aluno, id_disciplina)  
29 );
```

Questão 3 — FOREIGN KEY — Relacionamentos entre tabelas

A cláusula FOREIGN KEY cria um relacionamento entre tabelas, garantindo a integridade referencial entre os dados.

Exemplo:

```
CREATE TABLE pedido (  
    id_pedido INT PRIMARY KEY,  
    data_pedido DATE NOT NULL,  
    id_cliente INT,  
    FOREIGN KEY (id_cliente) REFERENCES cliente(id_cliente)  
);
```

Sua tarefa: Crie a tabela livro contendo id_livro (PK), titulo, ano_publicacao e id_autor, sendo este último uma chave estrangeira que referencia a tabela autor (suponha que autor já exista com id_autor como PK).

Resposta:

```
31 • CREATE TABLE livro (  
32     id_livro INT PRIMARY KEY,  
33     titulo VARCHAR(150),  
34     ano_publicacao INT,  
35     id_autor INT,  
36     FOREIGN KEY (id_autor) REFERENCES autor(id_autor)  
37 );
```

Questão 4 — ALTER TABLE — Adicionar coluna

O comando ALTER TABLE permite modificar a estrutura de uma tabela existente, como adicionar novas colunas.

Exemplo:

```
ALTER TABLE cliente
ADD COLUMN telefone VARCHAR(15);
```

Sua tarefa: Escreva o comando para adicionar uma coluna chamada data_nascimento (do tipo DATE) à tabela funcionario já existente.

Resposta:

```
39 • CREATE TABLE funcionario(
40     id_funcionario INT PRIMARY KEY,
41     nome VARCHAR(80) NOT NULL,
42     idade INT NOT NULL,
43     id_função VARCHAR(100) NOT NULL,
44     FOREIGN KEY (id_função) REFERENCES funcionario(id_funcionario)
45 );
46
47 • ALTER TABLE funcionario
48     ADD COLUMN data_nascimento DATE;
```

Questão 5 — ALTER TABLE — Modificar e remover coluna

Além de adicionar colunas, o ALTER TABLE permite alterar o tipo de dados de uma coluna existente ou removê-la da tabela.

Exemplo:

```
ALTER TABLE cliente
ALTER COLUMN nome TYPE VARCHAR(150);

ALTER TABLE cliente
DROP COLUMN telefone;
```

Sua tarefa: Na tabela produto, altere o tipo da coluna nome para VARCHAR(120) e, em seguida, remova a coluna descricao_antiga.

Resposta:

```
1 • CREATE TABLE produto (
2     id_produto INT PRIMARY KEY,
3     nome VARCHAR(80) NOT NULL,
4     preco DECIMAL(10,2),
5     quantidade_estoque INT
6 );
7
8 • ALTER TABLE produto
9     MODIFY COLUMN nome VARCHAR(120);
10
11 • ALTER TABLE produto
12     DROP COLUMN descricao_antiga;
```

Questão 6 — DROP TABLE — Remover tabela

O comando DROP TABLE remove permanentemente uma tabela do banco de dados, incluindo todos os seus dados e estrutura.

Exemplo:

```
DROP TABLE cliente;
```

Sua tarefa: Escreva o comando necessário para remover a tabela log_acesso do banco de dados. Em seguida, escreva uma versão que remove a tabela apenas se ela existir.

Resposta:

```
53 • DROP TABLE log_acesso;  
54 • DROP TABLE IF EXISTS log_acesso;
```

Questão 7 — CONSTRAINTS — CHECK e DEFAULT

As restrições CHECK e DEFAULT permitem validar valores e definir padrões para colunas de uma tabela.

Exemplo:

```
CREATE TABLE funcionario (  
    id_funcionario INT PRIMARY KEY,  
    nome VARCHAR(100) NOT NULL,  
    salario DECIMAL(10,2) CHECK (salario > 0),  
    ativo BOOLEAN DEFAULT TRUE  
);
```

Sua tarefa: Crie uma tabela conta_bancaria contendo id_conta (PK), titular, saldo (com restrição CHECK para que seja maior ou igual a zero) e tipo_conta (com valor padrão 'CORRENTE').

Resposta:

```
56 • CREATE TABLE conta_bancaria (  
57     id_conta INT PRIMARY KEY,  
58     titular VARCHAR(100) NOT NULL,  
59     saldo DECIMAL(12,2) CHECK (saldo >= 0),  
60     tipo_conta VARCHAR(20) DEFAULT 'CORRENTE'  
61 );
```

Questão 8 — CREATE INDEX — Criação de índices

O comando CREATE INDEX cria índices em colunas para melhorar o desempenho de consultas em tabelas grandes.

Exemplo:

```
CREATE INDEX idx_cliente_email  
ON cliente (email);
```

Sua tarefa: Escreva o comando para criar um índice chamado idx_produto_nome na coluna

nome da tabela produto e, em seguida, um índice UNIQUE chamado idx_cpf_funcionario na coluna cpf da tabela funcionario.

Resposta:

```
1 • CREATE TABLE produto (  
2     id_produto INT PRIMARY KEY,  
3     nome VARCHAR(80) NOT NULL,  
4     preco DECIMAL(10,2),  
5     quantidade_estoque INT  
6 );  
7  
8 • ALTER TABLE produto  
9     MODIFY COLUMN nome VARCHAR(120);  
10  
11 • ALTER TABLE produto  
12     DROP COLUMN descricao_antiga;  
13  
14 • CREATE INDEX idx_produto_nome  
15     ON produto (nome);
```

Questão 9 — CREATE VIEW — Criação de visões

O comando CREATE VIEW cria uma visão (consulta armazenada) que pode ser utilizada como se fosse uma tabela, facilitando consultas complexas.

Exemplo:

```
CREATE VIEW vw_clientes_ativos AS  
SELECT id_cliente, nome, email  
FROM cliente  
WHERE ativo = TRUE;
```

Sua tarefa: Crie uma view chamada vw_produtos_em_estoque que exiba id_produto, nome e quantidade dos produtos da tabela produto cuja quantidade em estoque seja maior que zero.

Resposta:

```
1 • CREATE TABLE produto (  
2     id_produto INT PRIMARY KEY,  
3     nome VARCHAR(80) NOT NULL,  
4     preco DECIMAL(10,2),  
5     quantidade_estoque INT  
6 );  
7  
8 • ALTER TABLE produto  
9     MODIFY COLUMN nome VARCHAR(120);  
10  
11 • ALTER TABLE produto  
12     DROP COLUMN descricao_antiga;  
13  
14 • CREATE INDEX idx_produto_nome  
15     ON produto (nome);  
16  
17 • CREATE VIEW vw_produtos_em_estoque AS  
18     SELECT id_produto, nome, quantidade_estoque  
19     FROM produto  
20     WHERE quantidade_estoque > 0;
```

Questão 10 — Modelagem completa — Aplicação integrada de DDL

Esta questão integra os conceitos vistos anteriormente, exigindo a criação de um pequeno modelo relacional completo utilizando comandos DDL.

Exemplo (mini-modelo de biblioteca):

```
CREATE TABLE editora (  
    id_editora INT PRIMARY KEY,  
    nome VARCHAR(100) NOT NULL  
);  
  
CREATE TABLE livro (  
    id_livro INT PRIMARY KEY,  
    titulo VARCHAR(150) NOT NULL,  
    ano INT CHECK (ano > 1500),  
    id_editora INT,  
    FOREIGN KEY (id_editora) REFERENCES editora(id_editora)  
);
```

Sua tarefa: Modele e escreva os comandos DDL para um sistema de escola contendo: (a) tabela curso (id_curso PK, nome); (b) tabela aluno (id_aluno PK, nome, email único, id_curso FK); (c) tabela disciplina (id_disciplina PK, nome, carga_horaria com CHECK > 0). Crie também um índice no email do aluno e uma view chamada vw_alunos_curso exibindo nome do aluno e nome do curso.

Resposta:

```
64 • CREATE TABLE curso (  
65     id_curso INT PRIMARY KEY,  
66     nome VARCHAR(100) NOT NULL  
67 );  
68  
69 • CREATE TABLE aluno (  
70     id_aluno INT PRIMARY KEY,  
71     nome VARCHAR(100) NOT NULL,  
72     email VARCHAR(80) UNIQUE,  
73     id_curso INT,  
74     FOREIGN KEY (id_curso) REFERENCES curso(id_curso)  
75 );  
77 • CREATE TABLE disciplina (  
78     id_disciplina INT PRIMARY KEY,  
79     nome VARCHAR(100) NOT NULL,  
80     carga_horaria INT CHECK (carga_horaria > 0)  
81 );  
82  
83 • CREATE UNIQUE INDEX idx_aluno_email  
84 ON aluno (email);  
85  
86 • CREATE VIEW vw_alunos_curso AS  
87 SELECT a.nome AS Nome_Aluno, c.nome AS Nome_Curso  
88 FROM aluno a  
89 INNER JOIN curso c  
90 ON a.id_curso = c.id_curso;
```

Bom estudo!